Assuming the block size is sensible, you don't have to worry about packaging the tasks, obtaining the futures, or storing the `std::thread` objects so you can join with the threads later; the thread pool takes care of that. All you need to do is call `submit()` with your task ❷.

The thread pool takes care of the exception safety too. Any exception thrown by the task gets propagated through the future returned from `submit()`, and if the function exits with an exception, the thread pool destructor abandons any not-yet-completed tasks and waits for the pool threads to finish.

This works well for simple cases like this, where the tasks are independent. But it's not so good for situations where the tasks depend on other tasks also submitted to the thread pool.

9.1.3. Tasks that wait for other tasks

The Quicksort algorithm is an example that I've used throughout this book. It's simple in concept: the data to be sorted is partitioned into those items that go before a pivot item and those that go after it in the sorted sequence. These two sets of items are recursively sorted and then stitched back together to form a fully sorted set. When parallelizing this algorithm, you need to ensure that these recursive calls make use of the available concurrency.

Back in chapter 4, when I first introduced this example, we used `std::async` to run one of the recursive calls at each stage, letting the library choose between running it on a new thread and running it synchronously when the relevant `get()` was called. This works well, because each task is either running on its own thread or will be invoked when required.

When we revisited the implementation in chapter 8, you saw an alternative structure that used a fixed number of threads related to the available hardware concurrency. In this case, you used a stack of pending chunks that needed sorting. As each thread partitioned the data it was sorting, it added a